

Fast-dVLM: Efficient Block-Diffusion VLM via Direct Conversion from Autoregressive VLM

Chengyue Wu^{1,2*} Shiyi Lan^{2*} Yonggan Fu² Sensen Gao⁴ Jin Wang^{1,2} Jincheng Yu²
Jose M. Alvarez² Pavlo Molchanov² Ping Luo¹ Song Han^{2,3} Ligeng Zhu^{2†} Enze Xie^{2†}

¹The University of Hong Kong ²NVIDIA ³MIT ⁴MBZUAI

*Equal contribution [†]Co-lead

Abstract: Vision-language models (VLMs) predominantly rely on autoregressive decoding, which generates tokens one at a time and fundamentally limits inference throughput. This limitation is especially acute in physical AI scenarios such as robotics and autonomous driving, where VLMs are deployed on edge devices at batch size one, making AR decoding memory-bandwidth-bound and leaving hardware parallelism underutilized. While block-wise discrete diffusion has shown promise for parallel text generation, extending it to VLMs remains challenging due to the need to jointly handle continuous visual representations and discrete text tokens while preserving pretrained multimodal capabilities. We present **Fast-dVLM**, a block-diffusion-based VLM that enables KV-cache-compatible parallel decoding and speculative block decoding for inference acceleration. We systematically compare two AR-to-diffusion conversion strategies: a *two-stage* approach that first adapts the LLM backbone with text-only diffusion fine-tuning before multimodal training, and a *direct* approach that converts the full AR VLM in one stage. Under comparable training budgets, direct conversion proves substantially more efficient by leveraging the already multimodally aligned VLM; we therefore adopt it as our recommended recipe. We introduce a suite of multimodal diffusion adaptations—block-size annealing, causal context attention, auto-truncation masking, and vision-efficient concatenation—that collectively enable effective block diffusion in the VLM setting. Extensive experiments across 11 multimodal benchmarks show Fast-dVLM matches its autoregressive counterpart in generation quality. With SGLang integration and FP8 quantization, Fast-dVLM achieves over 6× end-to-end inference speedup over the AR baseline.

1. Introduction

Vision-language models (VLMs) (Grattafiori et al., 2024; Wang et al., 2024; Bai et al., 2025) have become the foundation for a wide range of multimodal applications, from visual question answering to document understanding and chart interpretation. As these models are deployed in increasingly demanding settings, generating long-form reasoning chains, structured outputs, and multi-turn dialogues, inference efficiency becomes a critical bottleneck. Current VLMs almost universally rely on autoregressive (AR) decoding, producing tokens one at a time, which fundamentally limits generation throughput regardless of available hardware parallelism.

This efficiency gap is especially critical in physical AI, an increasingly important deployment frontier for VLMs. In robotics, autonomous driving, and embodied agents, VLMs serve as core perception-reasoning modules that must operate on resource-constrained edge platforms. Unlike cloud-serving workloads that amortize the cost of sequential decoding across large request batches, physical AI deployments are dominated by single-request, batch-size-one inference: each robot or vehicle processes its own observation stream independently. In this regime, AR decoding is fundamentally *memory-bandwidth-bound*—the model must load its full parameters for every generated token, yet utilizes only a small fraction of the available compute. Diffusion-based parallel decoding offers a natural remedy: by generating multiple tokens simultaneously within each block, it shifts the workload toward a more compute-bound regime and can therefore better exploit hardware parallelism, even under tight batch-size-one constraints.

Diffusion-based language models (Sahoo et al., 2024; Nie et al., 2025; Ye et al., 2025; Arriola et al., 2025a) have emerged as a promising alternative, enabling parallel multi-token decoding, with speculative decoding (Gao et al., 2025; Agrawal et al., 2025; Pan et al., 2025b; Chen et al., 2026; Pan et al., 2025a) providing further speedups. However, these advances remain largely confined to text-only settings.

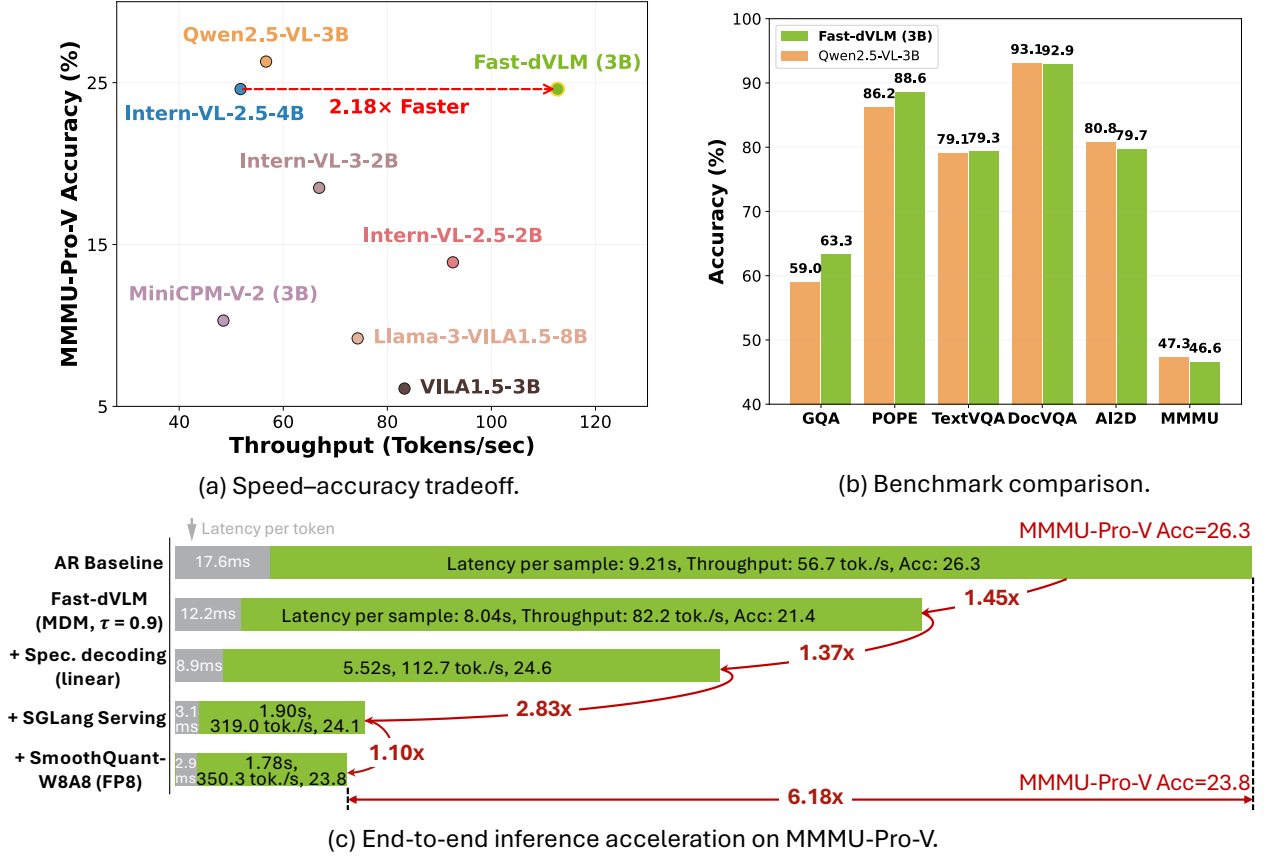


Figure 1 | Overview of our Fast-dVLM. (a) Fast-dVLM achieves comparable accuracy to AR VLM baselines on MMMU-Pro-V with a substantial speedup. (b) Benchmark comparison against the Qwen2.5-VL-3B backbone, showing near-lossless accuracy across diverse tasks. (c) Our combined approach achieves up to 6.18 $\times$ end-to-end speedup compared to the AR baseline. All throughput measurements are conducted on a single NVIDIA H100 GPU.

Extending diffusion generation to VLMs (You et al., 2025; Li et al., 2025b; Yang et al., 2025; Yu et al., 2025; Zeng et al., 2025; Arriola et al., 2025b; Cheng et al., 2025) is challenging, as the model must jointly handle visual and textual modalities while preserving pretrained multimodal capabilities. Early efforts (You et al., 2025; Li et al., 2025b; Yang et al., 2025; Yu et al., 2025) adopt full-sequence diffusion without block structure, precluding incremental KV caching; more recent works (Zeng et al., 2025; Arriola et al., 2025b; Cheng et al., 2025) introduce block diffusion with KV cache reuse. However, a key question remains underexplored: is it more effective to first adapt an AR LLM into a diffusion LLM and then fine-tune multimodally, or to directly convert a pretrained AR VLM in a single step?

In this work, we present **Fast-dVLM**, a block-diffusion-based vision-language model that, beyond KV-cache-compatible parallel decoding, further contributes self-speculative block decoding and system-level integration with SGLang (Zheng et al., 2024) for production-grade inference acceleration. At the core of Fast-dVLM is a systematic comparison of two AR-to-diffusion conversion strategies: a *two-stage* path that first converts the LLM backbone via text-only diffusion fine-tuning before multimodal fine-tuning, and a *direct* path that converts the full VLM in a single multimodal stage.

Our comparison reveals that, under comparable training budgets, direct conversion is significantly more training-efficient: by starting from a multimodally aligned VLM rather than a text-only LLM, it makes better use of the same data and compute. We hypothesize that both strategies share a similar performance ceiling but differ primarily in how efficiently they utilize the training budget, and adopt the direct path as our recommended recipe. Both paths build on Fast-dLLM v2 (Wu et al., 2025a) and extend it to VLMs with causal context attention, auto-truncation masking, vision-efficient concatenation, and self-speculative decoding for additional inference speedup.

Our contributions are threefold:

- We propose **Fast-dVLM**, a block-diffusion VLM compatible with KV caching and self-speculative decoding that achieves competitive quality with significant inference speedups over AR baselines.
- Through a controlled comparison of two-stage and direct AR-to-diffusion conversion, we find the direct path both simpler and more effective, and propose a systematic recipe—block-size annealing, auto-truncation masking, vision-efficient concatenation, and self-speculative decoding—each validated by ablation.
- Extensive experiments across 11 multimodal benchmarks show that Fast-dVLM matches or surpasses its AR counterpart. Combined with SGLang (Zheng et al., 2024) integration and FP8 quantization, Fast-dVLM achieves up to $6\times$ end-to-end inference speedup.

2. Related Work

Discrete diffusion language models have progressed from continuous-latent formulations (Lovelace et al., 2023; Strudel et al., 2022) to masked diffusion objectives (Austin et al., 2021; Lou et al., 2023; Sahoo et al., 2024), with LLaDA (Nie et al., 2025) and Dream (Ye et al., 2025) scaling to 7–8B parameters and matching AR baselines. Block-level generation (Arriola et al., 2025a; Wu et al., 2025b,a; Gat et al., 2025) further enables KV caching by autoregressively producing blocks whose tokens are denoised in parallel. Several concurrent works extend diffusion to VLMs—LLaDA-V (You et al., 2025), LaViDa (Li et al., 2025b), MMaDA (Yang et al., 2025), Dimple (Yu et al., 2025)—but all rely on full-sequence diffusion without block structure, precluding incremental KV caching and turn-aware attention masking. Speculative decoding for dLLMs (Gao et al., 2025; Agrawal et al., 2025; Pan et al., 2025b; Chen et al., 2026; Pan et al., 2025a; Li et al., 2025a) exploits native multi-token prediction for further speedup. Our work is the first to integrate block-level self-speculation into a multimodal diffusion VLM with system-level serving support. A detailed version is provided in Appendix D.

3. Methodology

3.1. Preliminary

Let $\mathbf{x} = (x_1, \dots, x_L)$ denote a token sequence of length L . Autoregressive models generate $\mathbf{x}$ by factoring $P_\theta(\mathbf{x}) = \prod_i P_\theta(x_i | x_{<i})$. Masked diffusion models (Sahoo et al., 2024; Nie et al., 2025) instead corrupt each token independently with masking probability $t \in (0, 1)$ and train a reverse model $p_\theta(\mathbf{x}_0 | \mathbf{x}_t)$ to recover masked tokens. Block-wise discrete diffusion (Arriola et al., 2025a; Wu et al., 2025a) partitions the sequence into blocks of size $\hat{B}$, generating blocks autoregressively while denoising tokens in parallel within each block, enabling KV-cache reuse across blocks.

Our model builds on Fast-dLLM v2 (Wu et al., 2025a), inheriting its complementary masking and dual-stream (noisy + clean) block attention design. Extending this text-only framework to VLMs raises four challenges: **(1) Conversion strategy**—should a pretrained AR VLM be converted via a two-stage text-first path or directly in one step? **(2) Multi-turn boundaries**—short responses (e.g., a single letter) cause the last denoising block to extend into the next turn’s prompt, leaking future information. **(3) Training efficiency**—the noisy-clean concatenation duplicates vision embeddings into both streams even though they are never corrupted, wasting memory and compute. **(4) Causal compatibility**—block-level context attention discards the pretrained causal structure and precludes AR decoding for self-speculative verification. The following subsections address each challenge.

3.2. AR-to-Diffusion Conversion Strategies

We compare two strategies for converting a pretrained AR VLM into a block-diffusion model (Figure 2):

Two-stage path. Stage 1 converts only the LLM backbone via text-only diffusion fine-tuning; Stage 2 attaches the vision encoder and MLP projector and jointly fine-tunes the entire model on multimodal data, with the vision encoder unfrozen since the LLM has already stabilized under the diffusion objective.

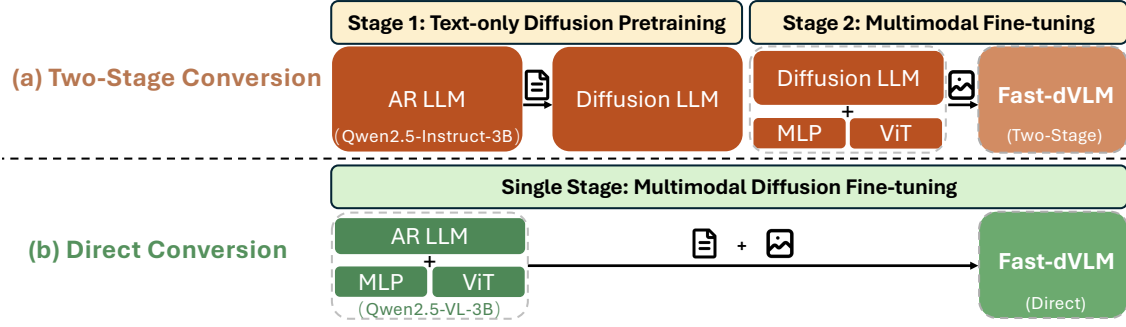


Figure 2 | Two AR-to-diffusion conversion strategies. (a) The *two-stage* strategy first converts the LLM backbone via text-only diffusion fine-tuning (Stage 1), then fine-tunes all components on multimodal data (Stage 2). (b) The *direct path* converts the full VLM in a single multimodal stage.

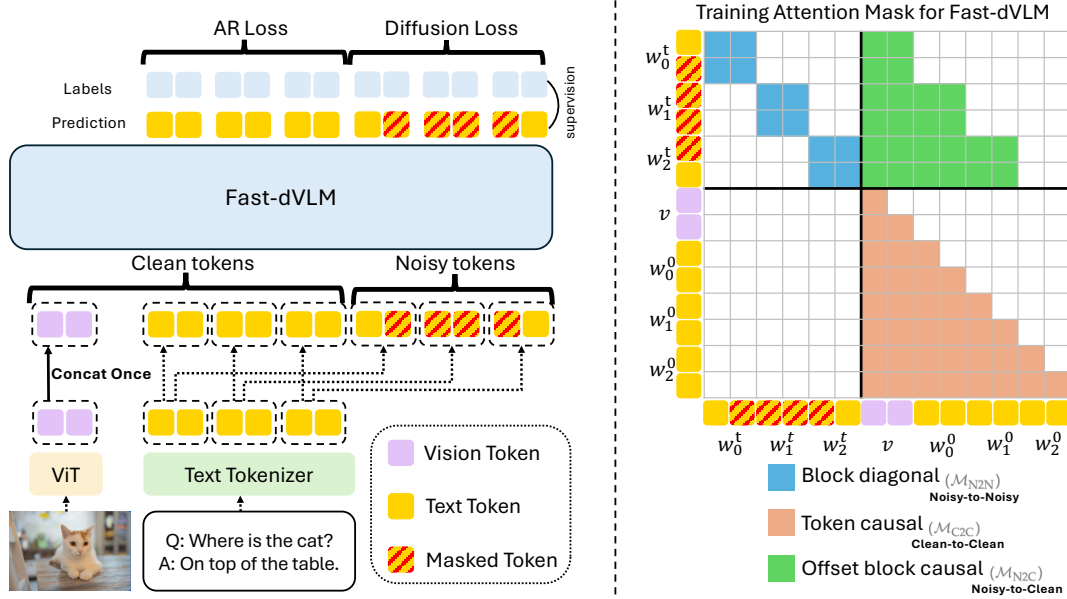


Figure 3 | Training architecture and attention mask for $[\mathbf{w}^t; \mathbf{x}]$ with block size $\hat{B} = 2$. The noisy stream $\mathbf{w}^t$ contains only text tokens; vision tokens appear exclusively in the clean stream $\mathbf{x}$ (vision-efficient concatenation). $\mathcal{M}_{N2N}$: block-diagonal attention for parallel denoising. $\mathcal{M}_{N2C}$: noisy tokens attend to preceding clean context including vision tokens. $\mathcal{M}_{C2C}$: token-level causal attention on the clean stream, enabling joint AR loss training and AR decoding at inference.

Direct path. The complete AR VLM is directly fine-tuned for block-diffusion on multimodal data in a single stage, yielding a simpler pipeline that leverages the pretrained multimodal alignment. Our experiments (Section 4) show that, under comparable training budgets, the direct path achieves substantially stronger performance, suggesting both strategies share a similar ceiling but the direct path makes more efficient use of its multimodal initialization.

3.3. Training

Both paths share the same training pipeline. Let $\mathbf{x} = (\mathbf{v}, \mathbf{w})$ denote the full input sequence, where $\mathbf{v}$ are vision token embeddings and $\mathbf{w}$ are text token embeddings. Only response text tokens are corrupted: a noisy stream $\mathbf{w}^t$ is constructed by masking response positions and concatenated with the clean stream to form $[\mathbf{w}^t; \mathbf{x}]$.

The attention mask (Figure 3) enforces three rules: noisy tokens attend bidirectionally within their block ($\mathcal{M}_{N2N}$); noisy tokens attend to clean tokens from preceding blocks ($\mathcal{M}_{N2C}$); clean tokens follow token-level causal attention ($\mathcal{M}_{C2C}$). Unlike Fast-dLLM v2’s block-level context attention, we use causal attention for all preceding context, which better preserves pretrained AR representations and supports AR decoding for self-speculative verification.

Block-size annealing. We adopt a curriculum that progressively increases the block size $\hat{B}$. Given candidate sizes $S = \{2^1, 2^2, \dots, B_d\}$ and training progress $u \in [0, 1]$, the active size is $\hat{B} = S[\min(\lfloor u \cdot |S| \rfloor, |S| - 1)]$, allowing the model to learn fine-grained denoising before tackling larger corruption spans.

Auto-truncation attention mask. In multi-turn dialogue, response lengths are rarely multiples of $\hat{B}$ —especially in multimodal data where responses can be extremely short (e.g., a single option letter). Without special handling, the last block would extend into the next turn’s prompt, and $\mathcal{M}_{N2N}$ would let noisy tokens attend to future prompt tokens. We address this by automatically truncating each response’s last block at the response boundary, preventing cross-turn leakage while preserving block-parallel denoising.

Vision-efficient concatenation. Since vision embeddings are never corrupted, they are identical across both streams. We therefore include them only in the clean stream; the noisy stream contains only text positions, and vision tokens are attended to via $\mathcal{M}_{N2C}$. On Qwen2.5-VL-3B (H100, context length 2048), this lossless design reduces peak memory by 15.0% and training time by 14.2%.

Training objective. Let W denote the language model head, and $\mathbf{H}^{(t)}, \mathbf{H}^{(0)}$ the hidden states of the noisy and clean streams. The total loss combines a diffusion loss with a causal LM loss ($\alpha = \beta = 0.5$):

$$\mathcal{L} = \alpha \text{CE}(W\mathbf{H}^{(t)}, \mathbf{y}) + \beta \text{CE}(W\mathbf{H}^{(0)}, \mathbf{y}),$$

where $\mathbf{y}$ are the response labels. The diffusion branch learns parallel denoising while the causal branch preserves AR generation capability.

3.4. Inference

As in Fast-dLLM v2 (Wu et al., 2025a), decoding proceeds block by block with KV-cache reuse; within each block, tokens are iteratively unmasked via confidence-aware parallel decoding (threshold τ). Our pipeline adds two key components: causal context decoding and self-speculative decoding.

Causal context decoding. Each block is seeded by a single AR step that generates the first token from the cached causal context; the remaining $\hat{B} - 1$ positions are filled with [MASK] and iteratively denoised, naturally aligning with the training-time causal attention pattern.

Self-speculative block decoding. We introduce a self-speculative variant (Samragh et al., 2025; Chen et al., 2026) where the diffusion mode drafts all $\hat{B} - 1$ tokens in one pass and the causal mode verifies them autoregressively, accepting the longest matching prefix and trimming the KV cache accordingly. We employ a *linear* scheme (two passes per block: draft + verify) and a *quadratic* scheme (Liu et al., 2025) that fuses verification and next-block proposal into one pass with $O(\hat{B}^2)$ input tokens. Details and pseudo code are in Appendix B.

System integration. We integrate Fast-dVLM into SGLang (Zheng et al., 2024), extending its scheduler to support alternating bidirectional-draft and causal-verify attention, enabling optimized kernels and CUDA graph for wall-clock speedups in production serving. We further incorporate SmoothQuant (Xiao et al., 2023) to enable W8A8 (FP8) quantization, reducing memory footprint and improving effective Tensor Core throughput.

4. Experiments

4.1. Experimental Setup

We initialize from Qwen2.5-VL-3B (Bai et al., 2025) and convert it via the direct path (Section 3.2) with all training recipes described in Section 3.3 ($B_d=32, \alpha=\beta=0.5$). At inference we evaluate MDM decoding and

Table 1 | Benchmark performance comparison (Part 1: short-answer benchmarks). Models are grouped into autoregressive (AR) and diffusion categories. Among diffusion models, **best** and 2nd best results are highlighted. MDM = masked diffusion model decoding; spec. = speculative decoding.

Model	<i>Short-answer Benchmarks</i>						
	AI2D	ChartQA	DocVQA	GQA	MMBench	MMMU	POPE
<i>Autoregressive Vision-Language Models</i>							
VILA-1.5-3B	58.0	53.0	44.3	61.4	60.5	31.8	86.8
MiniCPM-V-2 (3B)	65.0	59.2	69.8	51.7	66.3	37.9	86.5
Intern-VL-2.5-4B	81.3	77.8	91.1	61.0	80.7	50.0	89.3
Qwen2.5-VL-3B	80.8	84.0	93.1	59.0	76.9	47.3	86.2
<i>Diffusion Vision-Language Models</i>							
LaViDa	70.0	59.0	64.6	55.5	70.5	43.3	81.4
Dimple	74.4	63.3	37.7	59.2	<u>74.6</u>	45.2	<u>86.2</u>
LLaDA-V	<u>77.8</u>	<u>78.3</u>	<u>83.9</u>	53.4	82.9	48.6	81.8
Fast-dVLM (MDM)	79.7	82.8	92.1	<u>63.0</u>	74.2	44.6	88.6
Fast-dVLM (spec.)	79.7	83.1	92.9	63.3	74.3	<u>46.6</u>	88.6

Table 2 | Benchmark performance comparison (Part 2). Tokens/NFE measures average tokens decoded per forward pass. Among diffusion models, **best** and 2nd best results are highlighted. MDM = masked diffusion model decoding; spec. = speculative decoding.

Model	<i>Short-answer Benchmarks</i>				<i>Long-answer</i>	
	RealWorldQA	SEEDBench2+	TextVQA	Avg	MMMU-Pro-V	Tokens/NFE
<i>Autoregressive Vision-Language Models</i>						
VILA-1.5-3B	53.2	41.2	58.2	54.8	6.1	1.00
MiniCPM-V-2 (3B)	56.3	52.5	74.4	62.0	10.3	1.00
Intern-VL-2.5-4B	64.6	67.0	78.8	74.2	24.6	1.00
Qwen2.5-VL-3B	65.1	68.6	79.1	74.0	26.3	1.00
<i>Diffusion Vision-Language Models</i>						
LaViDa	54.5	57.7	60.3	61.7	10.5	1.00
Dimple	55.4	51.7	61.6	60.9	12.4	1.00
LLaDA-V	<u>63.2</u>	68.7	64.7	70.3	<u>18.6</u>	1.00
Fast-dVLM (MDM)	65.1	<u>67.2</u>	<u>76.1</u>	<u>73.3</u>	21.4	<u>1.95</u>
Fast-dVLM (spec.)	65.1	<u>67.2</u>	79.3	74.0	24.6	2.63

self-speculative decoding (linear variant by default; Algorithm 1). We benchmark on 11 tasks all evaluated with VLMEvalKit (Duan et al., 2024). Throughput (TPS) is measured on a single NVIDIA H100 GPU at batch size one, reflecting the single-request regime prevalent in physical AI deployments where AR decoding is memory-bandwidth-bound. We also report Tokens/NFE (average tokens decoded per forward pass; computed on MMMU-Pro-V samples with >200 response tokens). Full experimental details are in Appendix A.

4.2. Main Results

Table 1 compares Fast-dVLM (trained with $B_d = 32$) against AR and diffusion VLM baselines using two decoding strategies: MDM decoding and speculative (spec.) decoding.

Short-answer benchmarks. On short-answer tasks, Fast-dVLM achieves competitive results with the AR baseline while delivering significant inference speedup. With MDM decoding, the model reaches an average score of 73.3, closely matching the baseline’s 74.0, while achieving $1.95\times$ Tokens/NFE. It matches or surpasses the baseline on GQA (+4.0), POPE (+2.4), and RealWorldQA, suggesting that bidirectional context within each denoising block benefits holistic visual reasoning. With speculative decoding, the average score rises to 74.0, exactly matching the baseline, while Tokens/NFE reaches $2.63\times$. Among diffusion VLMs, Fast-dVLM achieves the best results on 8 out of 11 short-answer benchmarks, substantially outperforming prior diffusion

baselines.

Long-answer benchmarks. On MMMU-Pro-V, which requires multi-step chain-of-thought reasoning, the AR baseline scores 26.3. Fast-dVLM with MDM decoding scores 21.4, a gap of 4.9 points; speculative decoding narrows this to 24.6 (only 1.7 points behind). Long-form reasoning demands sequential coherence over many tokens, where block-wise parallel denoising is at a structural disadvantage. The remaining gap can likely be further narrowed with larger-scale training data and longer annealing schedules.

Speed-quality tradeoff. A clear and favorable tradeoff emerges across both strategies: MDM decoding provides $1.95\times$ Tokens/NFE with only a 0.7-point average quality drop on short-answer tasks, while speculative decoding achieves $2.63\times$ Tokens/NFE while exactly matching the baseline’s average quality (74.0). This confirms that block-wise discrete diffusion is a practical paradigm for VLMs, delivering acceleration without sacrificing benchmark accuracy on short-answer tasks.

4.3. Direct vs. Two-Stage Adaptation

As discussed in Section 3.2, we compare two AR-to-diffusion conversion strategies. The *two-stage* path starts from the AR LLM (Qwen2.5-Instruct-3B): it first fine-tunes on 300K text-only samples following the Fast-dLLM v2 recipe to obtain a diffusion LLM, then fine-tunes on multimodal data to produce a diffusion VLM. The *direct* path starts from the AR VLM (Qwen2.5-VL-3B) and converts it into a single multimodal fine-tuning stage. The multimodal fine-tuning data, training recipe, and compute budget are identical across both paths.

Figure 4 summarizes the comparison. The direct path achieves a substantially higher average score (73.3 vs. 60.2), outperforming the two-stage path on all 10 benchmarks. The gap is particularly large on knowledge- and reasoning-intensive tasks such as DocVQA (+31.5), ChartQA (+21.4), and AI2D (+18.1), where the pretrained multimodal alignment of the base VLM provides a strong advantage.

Notably, both paths consume **comparable training data** (2M training samples detailed in Appendix.A) and **compute budget** (both trained single epoch to avoid memorization), yet the direct path outperforms by a wide margin (radar chart in Figure.4). This is because the direct path inherits the multimodal alignment already acquired during VLM pretraining, making more efficient use of the same training budget, whereas the two-stage path starts from a text-only LLM and must rebuild this alignment from scratch. We hypothesize that both strategies share a similar performance ceiling but differ primarily in training efficiency, motivating our choice of the direct path as the default recipe.

4.4. Ablation Studies

We conduct ablation experiments under the direct conversion setting using the ShareGPT-4V dataset (Chen et al., 2023b). Table 3 isolates the contribution of each training recipe component by removing one at a time from the full configuration.

Causal context. Replacing the causal context attention with block-level bidirectional attention for all preceding context causes the most severe degradation, with average accuracy dropping by 22.5% (from 57.3 to 44.4). The damage is especially pronounced on reasoning-heavy benchmarks such as MMMU-Pro-V (−58.9%) and SeedBench2+ (−39.5%), confirming that causal context is essential for preserving the pretrained AR model’s sequential reasoning capability and enabling effective AR-loss co-training.

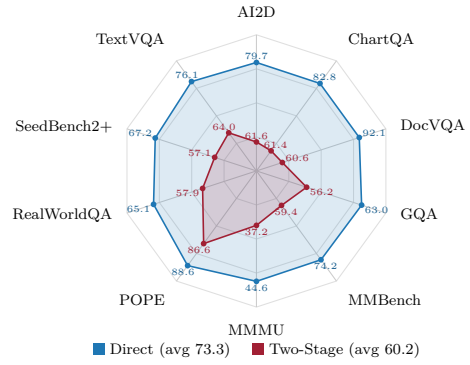


Figure 4 | Direct vs. two-stage adaptation across 10 benchmarks. Each axis is independently scaled.

Table 3 | Ablation study on training recipe. Each row removes one component from the full recipe. Parenthesized values show relative change: **red** for drops, **green** for gains.

Setting	MMBench	MMMU	POPE	MMMU-Pro-V	RealWorldQA	SeedBench2+	Avg
Full recipe	72.4	43.0	85.1	15.1	61.1	66.9	57.3
w/o causal context	58.5 (-19.2%)	29.9 (-30.5%)	71.1 (-16.5%)	6.2 (-58.9%)	60.0 (-1.8%)	40.5 (-39.5%)	44.4 (-22.5%)
w/o annealing block size	68.6 (-5.2%)	43.4 (+0.9%)	81.4 (-4.3%)	10.2 (-32.5%)	58.4 (-4.4%)	66.8 (-0.1%)	54.8 (-4.4%)
w/o auto-truncation	68.4 (-5.5%)	36.8 (-14.4%)	84.3 (-0.9%)	13.5 (-10.6%)	61.0 (-0.2%)	67.1 (+0.3%)	55.2 (-3.7%)

Annealing block size. Training directly with the target block size $B_d = 32$ without the curriculum schedule leads to a 4.4% average drop. The degradation is concentrated on long-form generation (MMMU-Pro-V -32.5%), indicating that progressive exposure to larger blocks is critical for the model to learn stable denoising at large corruption spans.

Auto-truncation attention mask. Removing the auto-truncation mechanism causes a 3.7% average drop, with a notable 14.4% decline on MMMU. Without truncation, the last block of each response extends into the next turn’s prompt, leaking future information during training and degrading generation reliability at inference time.

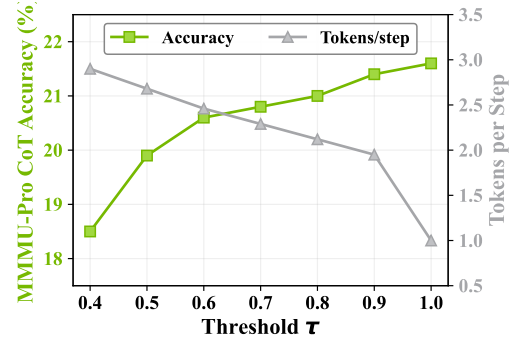
4.5. Inference Acceleration

We analyze three dimensions of inference acceleration: the confidence threshold τ that controls parallelism within MDM decoding, self-speculative decoding that further boosts throughput, and system-level integration with SGLang (Zheng et al., 2024) for production-grade serving. Figure 1(c) and Table 4 summarize wall-clock speedup on MMMU-Pro-V, where long-form generation makes latency savings most visible.

Decoding threshold. The confidence threshold τ governs the speed-quality tradeoff in MDM decoding (Figure 5). We sweep τ from 0.4 to 1.0: at $\tau = 1.0$, only one token is revealed per step (21.6 accuracy); relaxing to $\tau = 0.9$ nearly doubles throughput to 1.95 tokens/step while maintaining 21.4 accuracy; pushing to $\tau = 0.4$ maximizes parallelism at 2.90 tokens/step but accuracy drops to 18.5. We adopt $\tau = 0.9$ as the default for the best quality-speed balance.

Speculative block decoding. Self-speculative decoding further improves both quality and speed: the speculative variant recovers accuracy to 24.6 (close to the AR baseline’s 26.3) while achieving $1.98\times$ wall-clock TPS speedup (112.7 vs. 56.7 TPS). Figure 6 compares the linear and quadratic variants across block sizes. Tokens/NFE increases monotonically with block size for both variants, as larger blocks allow more tokens to be proposed per forward pass. The linear variant peaks in wall-clock TPS at block size 16 (112.7 TPS) and slightly drops at 32, as per-step computation grows while the acceptance rate saturates. The quadratic variant achieves higher Tokens/NFE thanks to its fused verify-and-propose design, but its TPS is lower at all block sizes: each step processes $\hat{B} \times (\hat{B} + 1)$ tokens with a non-standard attention mask pattern that current kernels are not optimized for, making the theoretical NFE advantage hard to realize in wall-clock time.

SGLang integration. We integrate Fast-dVLM into the SGLang inference engine, extending its scheduler with block-diffusion attention masking: the draft step uses bidirectional attention within each block and the verify step uses causal attention, both sharing the same paged KV cache. This allows Fast-dVLM to benefit

Figure 5 | Effect of threshold τ on MMMU-Pro CoT accuracy (left axis) and tokens per step (right axis).

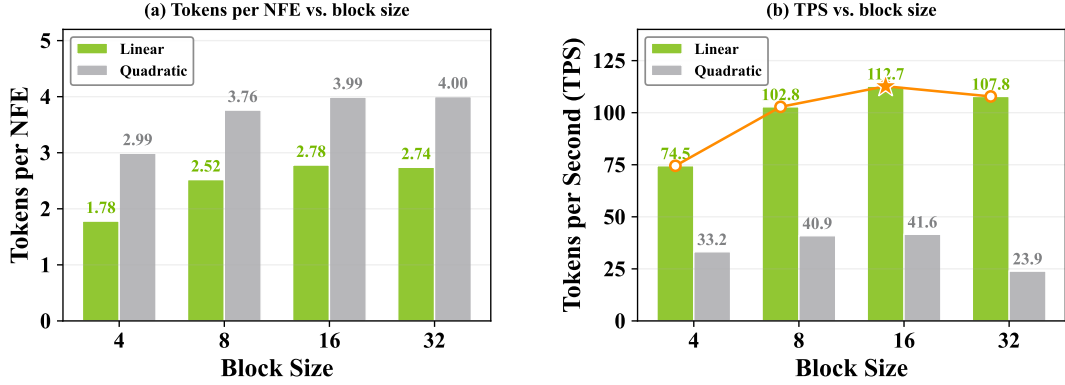


Figure 6 | Speculative decoding throughput across block sizes, comparing linear and quadratic variants. (a) Tokens per NFE increases with block size for both variants. (b) Wall-clock TPS peaks at block size 16 for both variants; the quadratic variant’s $O(B^2)$ cost becomes dominant at larger block sizes.

from SGLang’s optimized kernels, and CUDA graph. On top of speculative decoding, SGLang serving further boosts throughput to 319.0 TPS. To reduce memory footprint and better leverage Tensor Core efficiency, we further enable SmoothQuant W8A8 (FP8) quantization, increasing throughput to 350.3 TPS ($6.18\times$ speedup, Figure 1(c) and Table 4).

5. Conclusion

We presented Fast-dVLM, a block-diffusion vision-language model that converts a pretrained autoregressive VLM into diffusion VLM with KV-cache compatibility and self-speculative decoding. Through a systematic comparison of two AR-to-diffusion conversion strategies, we showed that direct conversion from a fully pretrained AR VLM is both simpler and more effective than the two-stage approach, better preserving the multimodal capabilities acquired during pretraining. We further proposed a comprehensive training recipe—including block-size annealing, causal context attention, auto-truncation masking, and vision-efficient concatenation—and validated the contribution of each component through ablation studies. Experiments across 11 multimodal benchmarks demonstrated that Fast-dVLM matches its AR counterpart. Combined with SGLang integration and FP8 quantization, Fast-dVLM achieves over $6\times$ end-to-end inference speedup.

References

- Sudhanshu Agrawal, Risheek Garrepalli, Raghavv Goel, Mingu Lee, Christopher Lott, and Fatih Porikli. Spiffy: Multiplying diffusion llm acceleration via lossless speculative decoding. *arXiv preprint arXiv:2509.18085*, 2025.
- Marianne Arriola, Aaron Gokaslan, Justin T. Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint arXiv:2503.09573*, 2025a.
- Marianne Arriola, Naveen Venkat, Jonathan Granskog, and Anastasis Germanidis. Adapting autoregressive vision language models for parallel diffusion decoding, 2025b. URL <https://runwayml.com/research/autoregressive-to-diffusion-vlms>.
- Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. *Advances in neural information processing systems*, 34: 17981–17993, 2021.
- Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.

- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023a.
- Jian Chen, Yesheng Liang, and Zhijian Liu. Dflash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.
- Jiaqi Chen, Jianheng Tang, Jinghui Qin, Xiaodan Liang, Lingbo Liu, Eric P Xing, and Liang Lin. Geoqa: A geometric question answering benchmark towards multimodal numerical reasoning. *arXiv preprint arXiv:2105.14517*, 2021.
- Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Pan, Conghui Song, and Yu Qiao. Sharegpt4v: Improving large multi-modal models with better captions. *arXiv preprint arXiv:2311.12793*, 2023b.
- Shuang Cheng, Yuhua Jiang, Zineng Zhou, Dawei Liu, Wang Tao, Linfeng Zhang, Biqing Qi, and Bowen Zhou. Sdar-vl: Stable and efficient block-wise diffusion for vision-language understanding. *arXiv preprint arXiv:2512.14068*, 2025.
- Haodong Duan, Xinyu Fang, Junming Yang, Xiangyu Zhao, Yuxuan Qiao, Mo Li, Amit Agarwal, Zhe Chen, Lin Chen, Yuan Liu, Yubo Ma, Hailong Sun, Yifan Zhang, Shiyin Lu, Tack Hwa Wong, Weiyun Wang, Peiheng Zhou, Xiaozhe Li, Chaoyou Fu, Junbo Cui, Jixuan Chen, Enxin Song, Song Mao, Shengyuan Ding, Tianhao Liang, Zicheng Zhang, Xiaoyi Dong, Yuhang Zang, Pan Zhang, Jiaqi Wang, Dahua Lin, and Kai Chen. Vlmevalkit: An open-source toolkit for evaluating large multi-modality models. *arXiv preprint arXiv:2407.11691*, 2024.
- Yifeng Gao, Ziang Ji, Yuxuan Wang, Biqing Qi, Hanlin Xu, and Linfeng Zhang. Self speculative decoding for diffusion large language models. *arXiv preprint arXiv:2510.04147*, 2025.
- Itai Gat, Heli Ben-Hamu, Marton Havasi, Daniel Haziza, Jeremy Reizenstein, Gabriel Synnaeve, David Lopez-Paz, Brian Karrer, and Yaron Lipman. Set block decoding is a language model inference accelerator. *arXiv preprint arXiv:2509.04185*, 2025.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Ishaan Gulrajani and Tatsunori B Hashimoto. Likelihood-based diffusion language models. *Advances in Neural Information Processing Systems*, 36:16693–16715, 2023.
- Drew A. Hudson and Christopher D. Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- Kushal Kafle, Scott Cohen, Brian Price, and Christopher Kanan. Dvqa: Understanding data visualizations via question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 5648–5656, 2018.
- Aniruddha Kembhavi, Mike Salvato, Eric Kolve, Minjoon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is worth a dozen images. In *European Conference on Computer Vision (ECCV)*, pp. 235–251, 2016.
- Geewook Kim, Teakgyu Hong, Moonbin Yim, JeongYeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, and Seunghyun Park. Ocr-free document understanding transformer. In *European Conference on Computer Vision*, pp. 498–517. Springer, 2022.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.
- Bohao Li, Yuying Ge, Yi Chen, Yixiao Ge, Ruimao Zhao, and Ying Shan. Seed-bench-2-plus: Benchmarking multimodal large language models with text-rich visual comprehension. *arXiv preprint arXiv:2404.16790*, 2024.

- Guanghao Li, Zhihui Fu, Min Fang, Qibin Zhao, Ming Tang, Chun Yuan, and Jun Wang. Diffuspec: Unlocking diffusion language models for speculative decoding. *arXiv preprint arXiv:2510.02358*, 2025a.
- Shufan Li, Konstantinos Kallidromitis, Hritik Bansal, Akash Gokul, Yusuke Kato, Kazuki Kozuka, Jason Kuen, Zhe Lin, Kai-Wei Chang, and Aditya Grover. Lavida: A large diffusion language model for multimodal understanding. *arXiv preprint arXiv:2505.16839*, 2025b.
- Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. Evaluating object hallucination in large vision-language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, 2023.
- Jingyu Liu, Xin Dong, Zhifan Ye, Rishabh Mehta, Yonggan Fu, Vartika Singh, Jan Kautz, Ce Zhang, and Pavlo Molchanov. Tidar: Think in diffusion, talk in autoregression. *arXiv preprint arXiv:2511.08923*, 2025.
- Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, et al. Mmbench: Is your multi-modal model an all-around player? In *European conference on computer vision*, pp. 216–233. Springer, 2024a.
- Zhijian Liu, Ligeng Zhu, Baifeng Shi, Zhuoyang Zhang, Yuming Lou, Shang Yang, Haocheng Xi, Shiyi Cao, Yuxian Gu, Dacheng Li, Xiuyu Li, Yunhao Fang, Yukang Chen, Cheng-Yu Hsieh, De-An Huang, An-Chieh Cheng, Vishwesh Nath, Jinyi Hu, Sifei Liu, Ranjay Krishna, Daguang Xu, Xiaolong Wang, Pavlo Molchanov, Jan Kautz, Hongxu Yin, Song Han, and Yao Lu. Nvila: Efficient frontier visual language models, 2024b. URL <https://arxiv.org/abs/2412.04468>.
- Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*, 2023.
- Justin Lovelace, Varsha Kishore, Chao Wan, Eliot Shekhtman, and Kilian Q Weinberger. Latent diffusion for language generation. *Advances in Neural Information Processing Systems*, 36:56998–57025, 2023.
- Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022*, pp. 2263–2279, 2022.
- Minesh Mathew, Dimosthenis Karatzas, and C.V. Jawahar. Docvqa: A dataset for vqa on document images. In *IEEE Winter Conference on Applications of Computer Vision (WACV)*, pp. 2200–2209, 2021.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.
- Jingyang Ou, Shen Nie, Kaiwen Xue, Fengqi Zhu, Jiacheng Sun, Zhenguo Li, and Chongxuan Li. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. *arXiv preprint arXiv:2406.03736*, 2024.
- Rui Pan, Zhuofu Chen, Hongyi Liu, Arvind Krishnamurthy, and Ravi Netravali. Fail fast, win big: Rethinking the drafting strategy in speculative decoding via diffusion llms. *arXiv preprint arXiv:2512.20573*, 2025a.
- Tianxiang Pan, Baitao Gong, Mo Guang, Hongwei Yong, Tianpeng Jiang, Yaqian Li, Zheng Cao, and Kaiwen Long. Blockspect: Blockwise speculative decoding for diffusion llms. *Submitted to ICLR 2026*, 2025b.
- Subham Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 37:130136–130184, 2024.
- Mohammad Samragh, Arnav Kundu, David Harrison, Kumari Nishu, Devang Naik, Minsik Cho, and Mehrdad Farajtabar. Your llm knows the future: Uncovering its multi-token prediction potential. *arXiv preprint arXiv:2507.11851*, 2025.

- Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. Towards vqa models that can read. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8317–8326, 2019.
- Robin Strudel, Corentin Tallec, Florent Alth  , Yilun Du, Yaroslav Ganin, Arthur Mensch, Will Grathwohl, Nikolay Savinov, Sander Dieleman, Laurent Sifre, et al. Self-conditioned embedding diffusion for text generation. *arXiv preprint arXiv:2211.04236*, 2022.
- Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.
- Chengyue Wu, Hao Zhang, Shuchen Xue, Shizhe Diao, Yonggan Fu, Zhijian Liu, Pavlo Molchanov, Ping Luo, Song Han, and Enze Xie. Fast-dllm v2: Efficient block-diffusion llm. *arXiv preprint arXiv:2509.26328*, 2025a.
- Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025b.
- xAI. Realworldqa, 2024. URL <https://huggingface.co/datasets/xai-org/RealWorldQA>.
- Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*, pp. 38087–38099. PMLR, 2023.
- Ling Yang, Ye Tian, Bowen Li, Xincheng Zhang, Ke Shen, Yunhai Tong, and Mengdi Wang. Mmada: Multimodal large diffusion language models. *arXiv preprint arXiv:2505.15809*, 2025.
- Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.
- Zebin You, Shen Nie, Xiaolu Zhang, Jun Hu, Jun Zhou, Zhiwu Lu, Ji-Rong Wen, and Chongxuan Li. Llada-v: Large language diffusion models with visual instruction tuning. *arXiv preprint arXiv:2505.16933*, 2025.
- Runpeng Yu, Xinyin Ma, and Xinchao Wang. Dimple: Discrete diffusion multimodal large language model with parallel decoding. *arXiv preprint arXiv:2505.16990*, 2025.
- Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, Cong Wei, Botao Yu, Ruibin Yuan, Renliang Sun, Ming Yin, Boyuan Zheng, Zhenzhu Yang, Yibo Liu, Wenhao Huang, Huan Sun, Yu Su, and Wenhui Chen. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9556–9567, 2024.
- Xiang Yue, Tianyu Zheng, Yuansheng Ni, Yubo Wang, Kai Zhang, Shengbang Tong, Yuxuan Sun, Botao Yu, Ge Zhang, Huan Sun, Yu Su, Wenhui Chen, and Graham Neubig. Mmmu-pro: A more robust multi-discipline multimodal understanding benchmark. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- Lunbin Zeng, Jingfeng Yao, Bencheng Liao, Hongyuan Tao, Wenyu Liu, and Xinggang Wang. Diffusionvl: Translating any autoregressive models into diffusion vision language models. *arXiv preprint arXiv:2512.15713*, 2025.
- Siyan Zhao, Devaansh Gupta, Qinqing Zheng, and Aditya Grover. d1: Scaling reasoning in diffusion large language models via reinforcement learning. *arXiv preprint arXiv:2504.12216*, 2025.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.

A. Experimental Setup

A.1. Training Data

Our multimodal fine-tuning stage is conducted on a diverse mixture of instruction-tuning datasets, curated in reference to the NVILA (Liu et al., 2024b) training mixture. This collection encompasses approximately 2 million training samples spanning general visual instruction tuning, document and chart understanding, as well as domain-specific scientific and geometric reasoning. Specifically, the dataset mixture consists of the following components:

- **General purpose and conversational data:** High-quality image-text instruction pairs sourced from ShareGPT4V (Chen et al., 2023b) (including the GPT-4 generated 100K split and the broader SFT subset) and LLaVA-Instruct (Liu et al., 2023).
- **Chart and data visualization:** Specialized subsets from DVQA (Kafle et al., 2018) and ChartQA (Masry et al., 2022).
- **Scientific and geometric reasoning:** Diagrams from AI2D (Kembhavi et al., 2016) and geometric problem-solving questions from GeoQA (Chen et al., 2021).
- **Document understanding:** Document images and text-rich visual question answering samples from DocVQA (Mathew et al., 2021) and synthetically generated document content via SynthDoG (Kim et al., 2022).

A.2. Training Configuration

We follow the direct path (Section 3.2), initializing from Qwen2.5-VL-3B (Bai et al., 2025). Training is conducted on 64 NVIDIA H100 GPUs (8 nodes $\times$ 8 GPUs per node) using DeepSpeed ZeRO Stage-2 with BF16 mixed precision and gradient checkpointing enabled. We train for 1 epoch with a cosine learning rate schedule, a peak learning rate of 5×10^{-6} , and a warmup ratio of 0.03. The per-device batch size is 1 with gradient accumulation over 4 steps, yielding an effective global batch size of 256. During training, the LLM backbone, vision encoder, and MLP projection layers are fine-tuned jointly. We use block-size annealing (Section 3.3) with a target block size of $B_d = 32$, complementary masking, causal context attention, and vision-efficient concatenation. The diffusion and causal loss branches are weighted equally ($\alpha = \beta = 0.5$).

A.3. Benchmarks

We evaluate on 11 VLM benchmarks spanning two categories. *Short-answer benchmarks* require brief, factual responses: AI2D (Kembhavi et al., 2016), ChartQA (Masry et al., 2022), DocVQA (Mathew et al., 2021), GQA (Hudson & Manning, 2019), MMBench (Liu et al., 2024a), MMMU (Yue et al., 2024), POPE (Li et al., 2023), RealWorldQA (xAI, 2024), SEEDBench2+ (Li et al., 2024), and TextVQA (Singh et al., 2019). *Long-answer benchmarks* require extended chain-of-thought reasoning: MMMU-Pro-V (Yue et al., 2025).

A.4. Evaluation Protocol

All benchmarks are evaluated using VLMEvalKit (Duan et al., 2024) under the same prompts and post-processing as the AR baseline. Throughput (tokens per second, TPS) is measured on a single NVIDIA H100 GPU at batch size one, reflecting the single-request regime prevalent in physical AI deployments (e.g., robotics and autonomous driving) where AR decoding is memory-bandwidth-bound. We also report Tokens/NFE (tokens per number of forward evaluations), which measures the average number of tokens decoded per forward pass. Tokens/NFE on MMMU-Pro-V is computed only on samples with response length greater than 200 tokens, as this metric is less meaningful for short responses. TPS speedup and Tokens/NFE are both reported relative to the AR baseline Qwen2.5-VL-3B.

B. Details of Speculative Decoding

We describe two self-speculative decoding strategies for block-causal diffusion VLMs. Both exploit the fact that our model supports two attention modes within a single set of weights: a *bidirectional* (diffusion) mode for parallel drafting and a *causal* (AR) mode for verification. The diffusion mode produces a draft of an

Table 4 | Inference acceleration on MMMU-Pro-V. Each row progressively adds one optimization on top of the previous. SpeedUp is relative to the AR baseline.

Setting	MMMU-Pro-V	TPS	SpeedUp
AR baseline	26.3	56.7	1.00×
Fast-dVLM (MDM, $\tau=0.9$)	21.4	82.2	1.45×
+ Spec. decoding (linear)	24.6	112.7	1.98×
+ SGLang serving	24.1	319.0	5.63×
+ SmoothQuant-W8A8 (FP8)	23.8	350.3	6.18×

Algorithm 1 Linear Speculative Block-Causal Decoding**Require:** prompt $\mathbf{x}_{1:L}$, block size B , mask token $[M]$

```

1: % Prefill (causal attention)
2:  $\mathbf{h}, \text{KV} \leftarrow \text{Forward}_{\text{causal}}(\mathbf{x}_{1:L})$ 
3:  $x_{L+1} \leftarrow \arg \max \mathbf{h}_L$ ;  $n \leftarrow L + 1$ 
4: while not done do
5:   % === Draft step: Bidirectional attention (Diffusion mode) ===
6:   % Each token in the block attends to all other tokens in the block,
7:   % and to all cached prefix tokens (via KV cache). No causal constraint.
8:    $\mathbf{d} \leftarrow [x_n, [M], \dots, [M]]$  ▷  $B$  tokens: 1 real +  $(B-1)$  masks
9:   Attention mask  $\mathbf{A}^{\text{draft}}: A_{ij} = 1 \ \forall i, j \in \{1, \dots, B\}$  ▷ Full bidirectional
10:   $\hat{\mathbf{h}} \leftarrow \text{Forward}(\mathbf{d}, \text{KV}, \mathbf{A}^{\text{draft}}, \text{no cache update})$ 
11:   $\hat{x}_i \leftarrow \arg \max \hat{\mathbf{h}}_{i-1}$  for  $i = 1, \dots, B$  ▷ Shift-by-one:  $\hat{\mathbf{h}}_{i-1}$  predicts position  $i$ 
12:   $d_i \leftarrow \hat{x}_i$  for all mask positions  $i$  ▷ Fill block with draft predictions
13:  % === Verify step: Causal attention (AR mode) ===
14:  % Standard left-to-right causal mask; each token only sees preceding tokens.
15:  Attention mask  $\mathbf{A}^{\text{verify}}: A_{ij} = \mathbb{I}[i \geq j]$  ▷ Causal (lower triangular)
16:   $\mathbf{h}^v, \text{KV} \leftarrow \text{Forward}(\mathbf{d}, \text{KV}, \mathbf{A}^{\text{verify}}, \text{update cache})$ 
17:   $a_i \leftarrow \arg \max \mathbf{h}_i^v$  for  $i = 0, \dots, B-1$  ▷ AR predictions
18:  % === Accept: left-to-right comparison ===
19:   $k \leftarrow \min\{j \in [0, B-2] : a_j \neq d_{j+1}\}$ ;  $k \leftarrow k + 1$  ▷ First mismatch; accept  $k$  tokens
20:  Append  $a_{0:k-1}$  to sequence;  $n \leftarrow n + k$ 
21:   $\text{KV} \leftarrow \text{Crop}(\text{KV}, n - 1)$  ▷ Discard rejected tokens from cache
22: end while
23: return generated sequence

```

entire block in one pass, and the causal mode verifies its consistency with the AR distribution—realizing the draft-then-verify paradigm within a single model.

Linear speculative decoding (Algorithm 1) processes each block with exactly two forward passes. In the *draft* step, $B-1$ mask tokens are appended after the last accepted token to form a block of size B , and a forward pass with bidirectional attention predicts all masked positions simultaneously. In the *verify* step, the filled block is re-evaluated with causal attention, yielding standard AR predictions. The draft and AR predictions are compared left to right: consecutive matches are accepted, and the first mismatch is replaced by the AR prediction. The KV cache is cropped to the accepted length before the next iteration. This requires 2 NFEs per block and can accept up to B tokens, giving a theoretical speedup of up to $B/2\times$ over AR decoding.

Quadratic speculative decoding (Algorithm 2) fuses verification and proposal into a single forward pass. The B draft tokens $[d_0, d_1, \dots, d_{B-1}]$ are expanded into B groups of $B+1$ tokens: group i is $[d_i, [M]^B]$, yielding a total input of $B(B+1)$ tokens. The attention mask ensures that (a) the first token of each group attends causally to first tokens of preceding groups (AR verification), (b) the B mask tokens within each

group attend bidirectionally to all tokens in the same group (parallel proposal), and (c) all tokens attend to the prefix via the KV cache. Position IDs for group i cover $[s+i, \dots, s+i+B]$, simulating an AR rollout from each acceptance point. After the forward pass, the output is reshaped to $B \times (B+1) \times |V|$: entry $(i, 0)$ verifies position $i+1$, while entries $(i, 1:B)$ provide the next-block proposal conditioned on accepting through position i . This requires only 1 NFE per block at the cost of $O(B^2)$ input tokens per step.

Trade-offs. The linear variant scales linearly with B and suits larger block sizes; the quadratic variant halves the number of forward passes but its $O(B^2)$ cost is more attractive for moderate B . Both variants produce the same token sequence as standard block-causal generation—the choice is purely a latency optimization depending on hardware and block size.

C. Case Study

We present qualitative examples to illustrate how Fast-dVLM (speculative decoding) compares with the AR baseline in terms of both response quality and decoding efficiency.

Math reasoning. Figure 7 shows a constrained optimization problem from MMMU-Pro-V. Both the AR baseline (Qwen2.5-VL-3B) and Fast-dVLM correctly identify the optimal point $(2, 1)$ and produce coherent step-by-step reasoning. Notably, the AR baseline renders its response in raw $\text{\LaTeX}$ markup (e.g., $\backslash($ and $\backslash\text{eq})$), whereas Fast-dVLM outputs clean, human-readable mathematical notation. Despite generating a comparable amount of reasoning, Fast-dVLM completes the response in 3 seconds at 77.7 tokens/s, a $1.6\times$ speedup over the baseline’s 5.4 seconds at 47.4 tokens/s.

Diverse task categories. Figure 8 provides additional examples spanning art style recognition, celebrity identification, and chart question answering. Across all three categories, Fast-dVLM generates detailed, accurate, and fluent responses: it correctly identifies the impressionist style and attributes the painting to Claude Monet; it recognizes Lionel Messi and provides extensive biographical context; and it comprehensively reads the chart data, identifies key trends, and discusses implications. Importantly, the decoding throughput remains high across different response lengths, ranging from 63.7 tokens/s for a short 120-token art description to 115.0 tokens/s for a long 636-token chart analysis, with Tokens/NFE ratios consistently above 1.5. These examples confirm that Fast-dVLM preserves the generation quality of the AR baseline while delivering substantial inference speedup across diverse visual understanding tasks.

Physical AI. Figure 9 demonstrates Fast-dVLM on two embodied AI scenarios. In the autonomous driving example, the model correctly reads highway signage and reasons about lane selection to reach Rochester, producing a concise 149-token response at 73.3 tokens/s. In the robotic manipulation example, the model generates a detailed 488-token, 8-step guide for picking up an industrial object and placing it into a bin, maintaining 73.0 tokens/s throughput. Both examples achieve a Tok./step ratio above 1.68, confirming that the block-diffusion speedup generalizes to long-form embodied reasoning tasks.

D. Extended Related Work

D.1. Diffusion LLMs

Discrete diffusion language models have evolved from continuous-latent formulations (Lovelace et al., 2023; Strudel et al., 2022) to discrete masked diffusion objectives (Austin et al., 2021; Lou et al., 2023; Ou et al., 2024; Gulrajani & Hashimoto, 2023; Sahoo et al., 2024), with recent models such as LLaDA (Nie et al., 2025) and Dream (Ye et al., 2025) scaling to 7–8B parameters and matching LLaMA-3 (Grattafiori et al., 2024) across reasoning benchmarks; d1 (Zhao et al., 2025) further shows that dLLMs benefit from reinforcement learning. A key remaining challenge is inference latency due to the incompatibility of bidirectional attention with KV caching. Block-level generation addresses this by autoregressively producing blocks whose internal tokens are denoised in parallel: Block Diffusion (Arriola et al., 2025a) enables KV caching and flexible-length generation;

Fast-dLLM (Wu et al., 2025b) and Fast-dLLM v2 (Wu et al., 2025a) introduce block-wise caching and efficient AR-to-diffusion adaptation recipes; and Set Block Decoding (Gat et al., 2025) combines next-token and masked-token prediction for 3–5 $\times$ fewer forward passes. With dLLMs now competitive on text-only tasks, extending this paradigm to vision-language settings is a natural next step.

D.2. Diffusion VLMs

Several concurrent works extend discrete diffusion to vision-language settings. LLaDA-V (You et al., 2025) projects visual features into a masked diffusion LLM and matches strong AR baselines such as LLaMA3-V (Grattafiori et al., 2024) and Qwen2-VL (Wang et al., 2024); LaViDa (Li et al., 2025b) introduces complementary masking and prefix KV caching to accelerate multimodal diffusion decoding; MMaDA (Yang et al., 2025) unifies multimodal reasoning and generation with a modality-agnostic architecture and RL-based post-training; and Dimple (Yu et al., 2025) adopts a hybrid AR-then-diffusion paradigm with confident decoding. However, all these models rely on full-sequence diffusion without block structure, precluding incremental KV caching of resolved response blocks, and none address turn-aware attention masking for multi-turn dialogue.

D.3. Speculative Decoding for Diffusion LLMs

Speculative decoding (Leviathan et al., 2023; Chen et al., 2023a) accelerates inference by drafting multiple tokens and verifying them in parallel. Recent work adapts this to dLLMs by exploiting their native multi-token prediction: SSD (Gao et al., 2025) uses the dLLM itself as both drafter and verifier; Spiffy (Agrawal et al., 2025) proposes auto-speculation via directed draft graphs; BlockSpec (Pan et al., 2025b) introduces block-level speculation with dynamic token exploration; DFlash (Chen et al., 2026) and FailFast (Pan et al., 2025a) integrate lightweight diffusion drafters; and DiffuSpec (Li et al., 2025a) shows that a pretrained dLLM can serve as a training-free drafter for AR verifiers. Our work integrates block-level self-speculation into a multimodal diffusion VLM for the first time.

Algorithm 2 Quadratic Speculative Block-Causal Decoding

Require: prompt $\mathbf{x}_{1:L}$, block size B , mask token $[\mathbf{M}]$

- 1: *% === Step 1: Draft-only forward ===*
- 2: *% Prompt tokens use causal attention; mask tokens attend to everything (bidirectional).*
- 3: $\mathbf{z} \leftarrow [\mathbf{x}_{1:L}, [\mathbf{M}]^B]$
- 4: Attention mask: $A_{ij} = \begin{cases} \mathbb{K}[i \geq j] & \text{if } i < L \text{ (prompt: causal)} \\ 1 & \text{if } i \geq L \text{ (mask block: attend to all)} \end{cases}$
- 5: $\mathbf{h}, \text{KV} \leftarrow \text{Forward}(\mathbf{z}, \mathbf{A})$
- 6: $d_i^{(1)} \leftarrow \arg \max \mathbf{h}_{L-1+i}$ for $i = 0, \dots, B-1$ ▷ First block draft
- 7: $\text{KV} \leftarrow \text{Crop}(\text{KV}, L)$ ▷ Only keep prompt cache
- 8: *% === Step 2: Quadratic verify + propose loop ===*
- 9: $s \leftarrow 0$ ▷ Total accepted tokens
- 10: **while** $s < T$ **do**
- 11: *% Construct quadratic input: expand B draft tokens into B groups of (B+1)*
- 12: Given draft block $\mathbf{d} = [d_0, d_1, \dots, d_{B-1}]$
- 13: Build input: $\mathbf{q} = [\underbrace{d_0, [\mathbf{M}]^B}_{\text{group 0}}, \underbrace{d_1, [\mathbf{M}]^B}_{\text{group 1}}, \dots, \underbrace{d_{B-1}, [\mathbf{M}]^B}_{\text{group } B-1}]$
- 14: ▷ Total length: $B \times (B+1)$
- 15: *% Position IDs: group i gets positions $[s+i, s+i+1, \dots, s+i+B]$*
- 16: *% This simulates "if accepted up to d_i , generate next B tokens"*
- 17: *% Attention mask structure (within the $B(B+1)$ query tokens):*
- 18: *% (a) All tokens attend to entire KV cache (prefix)*
- 19: *% (b) First token of each group (d_i) attends causally to first tokens of groups $\leq i$*
- 20: *% (c) Non-first tokens in group i attend to all tokens in the same group (bidirectional)*
- 21: *% (d) No cross-group attention for non-first tokens*
- 22: $\mathbf{H}, \text{KV}' \leftarrow \text{Forward}(\mathbf{q}, \text{KV}, \mathbf{A}^{\text{quad}})$
- 23: Reshape: $\mathbf{H} \in \mathbb{R}^{B \times (B+1) \times |V|}$
- 24: $P_{i,j} \leftarrow \arg \max \mathbf{H}_{i,j}$ ▷ Group i , position j
- 25: *% Verify: $P_{i,0}$ is the AR prediction for position $s+i+1$*
- 26: $k \leftarrow 1$
- 27: $\mathbf{d}^{\text{next}} \leftarrow P_{0,1:B}$ ▷ Default: next-block proposal from group 0
- 28: **while** $k < B$ **and** $P_{k-1,0} = d_k$ **do**
- 29: $\mathbf{d}^{\text{next}} \leftarrow P_{k,1:B}$ ▷ Update proposal from latest accepted group
- 30: $k \leftarrow k + 1$
- 31: **end while**
- 32: Accept $d_{0:k-1}$; $s \leftarrow s + k$
- 33: $\mathbf{d} \leftarrow \mathbf{d}^{\text{next}}$ ▷ Next block's draft
- 34: $\text{KV} \leftarrow \text{extract draft-token-only KVs from KV}'$ ▷ Keep every $(B+1)$ -th entry
- 35: **end while**
- 36: **return** $\mathbf{x}_{1:L+s}$

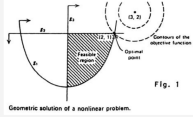
User: Answer the following multiple-choice question in the image. Think step by step before answering.

Qwen2.5-VL-3B (5.4 secs, 47.4 tokens/s): The image presents a mathematical problem and its solution. The problem is to minimize the function $\sqrt{(x_1 - 3)^2 + (x_2 - 2)^2}$ subject to the constraints $\sqrt{x_1^2 - x_2 - 3} \leq 0$, $\sqrt{x_2 - 1} \leq 0$, and $\sqrt{-x_1} \leq 0$. The solution is illustrated in a graph, labeled as "Fig. 1," which shows the feasible region and the contours of the objective function.

No. 15: Consider the following problem:

Minimize $(x_1 - 3)^2 + (x_2 - 2)^2$ subject to $x_1^2 - x_2 - 3 \leq 0$, $x_2 - 1 \leq 0$, $-x_1 \leq 0$. Give the solution.

[image 1]



Options:

- | | |
|------------|------------|
| (A) (2, 1) | (B) (1, 1) |
| (C) (1, 0) | (D) (3, 2) |
| (E) (2, 0) | (F) (2, 2) |
| (G) (3, 3) | (H) (3, 1) |
| (I) (0, 0) | (J) (0, 2) |

The options provided are:

- (A) (2, 1)
- (B) (1, 1)
- (C) (1, 0)
- (D) (3, 2)
- (E) (2, 0)
- (F) (2, 2)
- (G) (3, 3)
- (H) (3, 1)
- (I) (0, 0)

The correct answer is (A) (2, 1).

Fast-dVLM (3 secs, 77.7 tokens/s): To minimize the function $(x_1 - 3)^2 + (x_2 - 2)^2$, we need to find the point where the function is at its lowest value. This point is the optimal solution to the problem.

The given problem has two constraints: $x_1^2 - x_2 - 3 \leq 0$ and $x_2 - 1 \leq 0$. These constraints define a feasible region in the x_1 - x_2 plane.

The objective function $(x_1 - 3)^2 + (x_2 - 2)^2$ is a paraboloid that opens upwards. The lowest point on this paraboloid is the point where the function is minimized.

To find the optimal solution, we can use geometric methods. The optimal solution is the point where the objective function is at its lowest value, and it must lie within the feasible region.

From the figure, we can see that the optimal solution is the point (2, 1). This point lies within the feasible region and is the lowest point on the paraboloid.

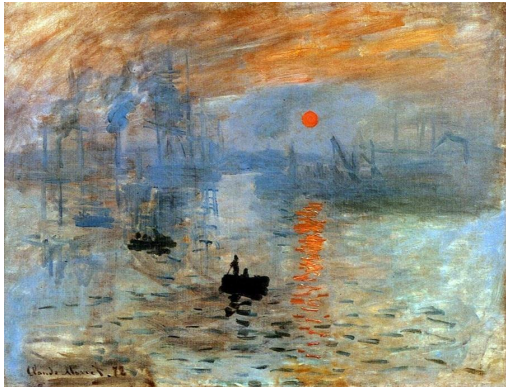
Therefore, the correct answer is (A) (2, 1).

Answer: (A)

Figure 7 | Qualitative comparison on an MMMU-Pro-V math reasoning problem. Both the AR baseline (Qwen2.5-VL-3B) and Fast-dVLM (speculative decoding) arrive at the correct answer, while Fast-dVLM achieves 1.6 $\times$ faster decoding speed (77.7 vs. 47.4 tokens/s).

Art

Gen. Len= 120, Tok./sec: 63.7, Tok./step: 1.52



The painting style of the image is impressionism, and the artist is Claude Monet. The painting depicts a cityscape with a large orange sun in the sky, reflecting on the water. The city is filled with buildings, and there are people walking on the streets. The painting is done in a loose, sketchy style, with visible brushstrokes and a focus on capturing the overall impression of the scene rather than the minute details. The use of color and light is characteristic of Monet's impressionist style, which aims to capture the fleeting effects of light and color in a scene.

Celebrity

Gen. Len= 260, Tok./sec: 59.6, Tok./step: 1.41



The player in the image is Lionel Messi, a renowned Argentine professional footballer. Born on June 24, 1987, in Rosario, Argentina, Messi has made an indelible mark on the world of football. He is widely regarded as one of the greatest football players of all time, known for his exceptional skills, creativity, and goal-scoring ability.

Messi began his professional career with Barcelona in 2004, where he has spent the majority of his career. He is a key player for the club, known for his ability to control the game and create opportunities for his teammates. His iconic number 10 jersey has become synonymous with Barcelona's attacking style.

Beyond Barcelona, Messi has represented the Argentina national team, winning numerous titles and accolades. He has been a consistent performer, scoring goals and providing assists in various international tournaments. His influence extends beyond the field, as he is a global icon and a symbol of passion, determination, and excellence in the sport of football.

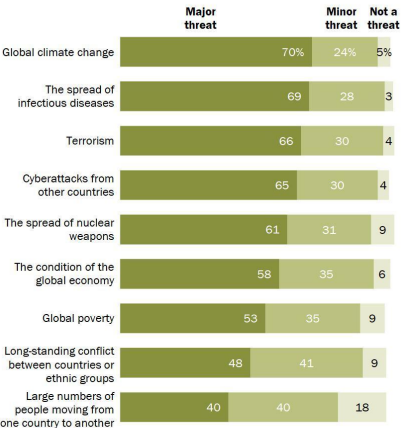
In summary, the player in the image is Lionel Messi, an Argentine footballer who has made significant contributions to both Barcelona and the Argentina national team. His talent, dedication, and impact on the sport have earned him a place among the greatest players in history.

Chart QA

Gen. Len= 636, Tok./sec: 115.0, Tok./step: 2.60

Across 14 countries polled, climate change and infectious diseases top list of global threats

Median % who say the following are a ___ to their country



Note: Percentages are medians based on 14 countries surveyed: U.S., Canada, Belgium, Denmark, France, Germany, Italy, Netherlands, Spain, Sweden, UK, Australia, Japan and South Korea. Those who did not answer are not shown.
Source: Spring 2020 Global Attitudes Survey, Q13a1.
"Despite Pandemic, Many Europeans Still See Climate Change as Greatest Threat to Their Countries"

PEW RESEARCH CENTER

The chart is a horizontal bar graph that presents the median percentage of people across 14 countries who consider various global threats as major, minor, or not a threat. The title of the chart is "Across 14 countries polled, climate change and infectious diseases top list of global threats." The data points visible in the chart are:

1. Global climate change: 70% major threat, 24% minor threat, 5% not a threat.
2. The spread of infectious diseases: 69% major threat, 28% minor threat, 3% not a threat.
3. Terrorism: 66% major threat, 30% minor threat, 4% not a threat.
4. Cyberattacks from other countries: 65% major threat, 30% minor threat, 4% not a threat.
5. The spread of nuclear weapons: 61% major threat, 31% minor threat, 9% not a threat.
6. The condition of the global economy: 58% major threat, 35% minor threat, 6% not a threat.
7. Global poverty: 53% major threat, 35% minor threat, 9% not a threat.
8. Long-standing conflict between countries or ethnic groups: 48% major threat, 41% minor threat, 9% not a threat.
9. Large numbers of people moving from one country to another: 40% major threat, 40% minor threat, 18% not a threat.

Key trends in the data include:

1. Climate change and infectious diseases are consistently ranked as major threats across all 14 countries.
2. Terrorism, cyberattacks, and nuclear weapons are also considered major threats, but to a lesser extent than climate change and infectious diseases.
3. The condition of the global economy, global poverty, and long-standing conflict are considered major threats in some countries but not in others.
4. Large numbers of people moving from one country to another are considered a major threat in some countries but not in others.

Outliers in the data include:

1. The condition of the global economy, global poverty, and long-standing conflict are considered major threats in some countries but not in others, which may indicate that these issues are more or less pressing in certain countries.
2. Large numbers of people moving from one country to another are considered a major threat in some countries but not in others, which may indicate that the issue is more or less relevant in certain countries.

Implications of the data include:

1. Climate change and infectious diseases are significant global threats that require urgent action from all countries.
2. Terrorism, cyberattacks, and nuclear weapons are also major threats that need to be addressed through international cooperation and policy measures.
3. The condition of the global economy, global poverty, and long-standing conflict are important issues that require attention and resources from all countries.
4. Large numbers of people moving from one country to another are a growing global challenge that requires international cooperation and policy measures to address.

Autonomous Driving	Gen. Len= 149, Tok./sec: 73.3, Tok./step: 1.71
	Sure, I'd be happy to help you with that! Here's a step-by-step approach to determine which lane to stay in to reach Rochester: 1. Identify the exit you want to take: In this case, you want to go to Rochester. 2. Look for the exit sign: The exit sign for Rochester is located in the middle of the image. 3. Observe the arrows on the exit sign: The arrows on the exit sign point to the left, right, and middle lanes. 4. Determine the correct lane: Since you want to go to Rochester, you should stay in the middle lane, as the arrows point to it. So, to summarize, to reach Rochester, you should stay in the middle lane.
Physical AI	Gen. Len= 488, Tok./sec: 73.0, Tok./step: 1.68
	Sure, I'd be happy to help you with that! Here's a step-by-step guide on how to pick up an industry object and place it in the yellow bin: 1. Identify the industry object: First, you need to locate the industry object on the table. It could be any small item, such as a screw, a bolt, or a small piece of hardware. Make sure you can see it clearly and that it's within reach. 2. Decide which hand to use: You have two robotic hands, one on each side of the table. Decide which hand you want to use to pick up the industry object. It's usually best to use the hand that's not currently holding anything, to avoid confusion. 3. Move the hand towards the industry object: Slowly move the chosen hand towards the industry object, keeping it steady and in control. Make sure you're not applying too much pressure, as this could damage the object or the robot's hand. 4. Grasp the industry object: When the hand is close enough to the industry object, gently apply pressure to the object with the fingers of the robot's hand. You may need to adjust the grip or angle of the hand to get a good grasp. Be careful not to apply too much force, as this could damage the object or the robot's hand. 5. Lift the industry object: Once you have a good grasp on the industry object, slowly lift it off the table. Make sure you're not applying any unnecessary force, and that the object is stable and secure in the hand. 6. Move the hand towards the yellow bin: Carefully move the hand with the industry object towards the yellow bin. Make sure you're not applying any force or movement to the object, as this could cause it to fall out of the hand. 7. Release the industry object: When the hand is in position, gently release the industry object into the yellow bin. Make sure you're not applying any force or movement to the object, as this could cause it to fall out of the bin or damage the bin. 8. Clean up: Once the industry object is in the yellow bin, clean up any debris or waste from the table. Make sure you're following any safety guidelines or procedures for handling industrial materials. By following these steps, you can safely and effectively pick up an industry object and place it in the yellow bin using your robotic hands.

Figure 9 | Qualitative examples of Fast-dVLM (speculative decoding) on embodied and physical AI tasks: autonomous driving scene understanding and robotic manipulation instruction. For each example, we report the generated response length (Gen. Len), decoding throughput (Tok./sec), and tokens per decoding step (Tok./step). Fast-dVLM produces detailed, step-by-step reasoning for both tasks while maintaining high throughput (~ 73 tokens/s) and a Tok./step ratio above 1.68.